

Documentazione AutoDocker

Documentazione Codice

Gruppo LZO: Vittorio Pedroni, Adrienne Macazar

Introduzione

In questo documento verrà presentato il codice del progetto con tutte le spiegazioni annesse. Ogni sezione rappresenterà una directory, mentre le sottosezioni rappresenteranno ogni file di codice. Qui sotto viene elencata la struttura generale delle directory e dei file più importanti:

- apache conf:

- ├── 000-default.conf
- └── ports.conf

- backend:

- ├── Dockerfiles
- ├── dockerfilePresets
- ├── compose-generator.php
- └── dockerfile-generator.php

- frontend:

- ├── assets:
 - │ └── css:
 - │ │ ├── dockercompose-edit.css
 - │ │ ├── dockerfile-gen-style.css
 - │ │ ├── footer-style.css
 - │ │ ├── index-style.css
 - │ │ └── navbar-style.css
 - │ └── html:
 - │ │ ├── dockercompose-edit.html
 - │ │ ├── dockerfile-form-view.html
 - │ │ ├── dockerfile-preview-view.html
 - │ │ ├── footer-view.html
 - │ │ ├── home-cards.html
 - │ │ ├── home-descriptions.html
 - │ │ └── navbar-view.html
 - └── pages:
 - │ └── dockerfile-gen.html
 - └── script:
 - │ ├── buttonAction.js
 - │ ├── dockerComposeDiv.js
 - │ ├── dockerfile-gen-presenter.js
 - │ ├── index-presenter.js
 - │ ├── inputHandler.js
 - │ ├── pageTheme.js
 - │ ├── sortable-container-init.js
 - │ └── submitHandler.js
- └── index.html

- Dockerfile

- docker-compose.yml

1. Dockerfile

Questo file contiene le istruzioni necessarie per costruire l'immagine per il nostro container *web-server*.
Spiegazione dei vari comandi:

- **FROM ubuntu:latest** : indica a Docker di partire con l'immagine base più recente di ubuntu presente su DockerHub;
- **ENV DEBIAN_FRONTEND=noninteractive** : Evita l'attesa dell'input utente durante *apt-get*;
- **ENV TZ=EUROPE/Rome** : inizializza il timezone del container a Europa/Roma;
- **RUN apt-get update && apt-get install -y php apache2** : indica a Docker di installare *php* e *apache*;
- **RUN mkdir /var/www/api** : crea nel container la directory */var/www/api*;
- **COPY ...** : copia i file presenti sulla macchina host sul path indicato dentro il nuovo path specificato per il container;
- **EXPOSE 80, EXPOSE 8080** : indica che il container sarà in ascolto sulle porte 80 e 8080;
- **CMD ["apache2ctl", "-D", "FOREGROUND"]** : dopo aver avviato il container, verrà eseguito questo comando per avviare apache nel foreground;

```
FROM ubuntu:latest

ENV DEBIAN_FRONTEND=noninteractive
ENV TZ=Europe/Rome

RUN apt-get update && apt-get install -y php apache2
RUN mkdir /var/www/api/
RUN a2enmod rewrite && a2enmod headers

COPY ./apache-conf/000-default.conf \
      /etc/apache2/sites-available/000-default.conf
COPY ./apache-conf/ports.conf /etc/apache2/ports.conf

EXPOSE 80
EXPOSE 8080

CMD ["apache2ctl", "-D", "FOREGROUND"]
```

2. docker-compose.yml

Questo file indica a Docker come configurare ed inizializzare i diversi servizi forniti dal container.

- **Networks:** viene definita una bridge chiamata *dev-bridge*, la quale collegherà i container presenti sulla rete *172.21.0.0/24*;
- **Services:** in questa sezione vengono definiti i servizi da creare. Verrà creato il container *web-server* partendo dall'immagine creata dal Dockerfile. Verranno montati i volumi */var/www/html* e */var/www/api* con i contenuti delle directory di frontend e backend presenti sulla macchina host. Le porte 80 e 8080 dell'host e del container vengono messe in comunicazione tra di loro. Infine, il container verrà messo in collegamento con la network *dev-bridge*.

```
networks:
  dev-bridge:
    driver: bridge
    name: dev-bridge
    ipam:
      config:
        - subnet: 172.21.0.0/24

services:
  web-server:
    container_name: web-server
    build:
      context: .
      dockerfile: Dockerfile
    volumes:
      - ./frontend:/var/www/html/
      - ./backend:/var/www/api/
    ports:
      - "80:80"
      - "8080:8080"
    networks:
      dev-bridge:
        ipv4_address: 172.21.0.2
```

3. apache-conf

3.1 000-default.conf

Questo file contiene le configurazioni necessarie per i virtual host di frontend e backend.

a) **Frontend <VirtualHost *:80>**

Inizializza la directory di root in /var/www/html nel container.

I log di errore verranno scritti su error_80.log, mentre i log generali verranno scritti su access_log.80.

Quest'ultimo conterrà informazioni più dettagliate dato che ha l'opzione *combined*;

b) **Backend <VirtualHost *:8080>**

Inizializza la directory di root in /var/www/api nel container.

Vengono applicate le opzioni seguenti opzioni:

- **Options Indexes:** se si accede ad una directory che non contiene un file *index*, viene mostrato la lista di file presenti nella directory;
- **Follow SymLinks:** permette ad Apache di seguire i link simbolici che portano ad altre directory o file;
- **Allow Override All:** permette l'utilizzo del file .htaccess nella directory per poter sovrascrivere le configurazioni di default di Apache;
- **Require all granted:** permette l'accesso alla directory a tutti quelli che vogliono accedere;
- **Header set Access-Control-Allow-Origin "*":** imposta l'header necessario per permettere ai CORS di accedere alla directory. Qualsiasi dominio può dunque accedere.

I log di errore verranno scritti su error_8080.log, mentre quelli generali su access_8080.log in modalità combined.

```
1 <VirtualHost *:80>
2     DocumentRoot /var/www/html
3     ServerName dockerfile-gui
4
5     ErrorLog ${APACHE_LOG_DIR}/error_80.log
6     CustomLog ${APACHE_LOG_DIR}/access_80.log combined
7 </VirtualHost>
8
9 <VirtualHost *:8080>
10     DocumentRoot /var/www/api
11     <Directory /var/www/api>
12         Options Indexes FollowSymLinks
13         AllowOverride All
14         Require all granted
15         Header set Access-Control-Allow-Origin "*"
16     </Directory>
17
18     ErrorLog ${APACHE_LOG_DIR}/error_8080.log
19     CustomLog ${APACHE_LOG_DIR}/access_8080.log combined
20 </VirtualHost>
```

3.2 ports.conf

Questo file indica le porte su cui il server deve ascoltare: 80 e 8080.

```
1 Listen 80
2 Listen 8080
```

4. Backend

4.1 dockerfilePresets

Contiene i dati di preset dei Dockerfile più utilizzati in formato JSON.

Al momento sono presenti 3 preset: NodeJS, Flask ed Apache.

Questi JSON vengono usati per riempire i campi di input del form automaticamente quando l'utente vuole usufruirne.

JSON NODEJS:

```
1 {  
2   "from": "node:14",  
3   "workdir": "/usr/src/app",  
4   "copy": ". .",  
5   "run": "npm install",  
6   "expose": "3000",  
7   "cmd": "[\"npm\", \"start\"]"  
8 }
```

JSON FLASK:

```
1 {  
2   "from": "python:3.8",  
3   "workdir": "/usr/src/app",  
4   "copy": ". .",  
5   "run": "pip install --no-cache-dir -r requirements.txt",  
6   "expose": "5000",  
7   "cmd": "[\"python\", \"app.py\"]"  
8 }
```

JSON APACHE:

```
1 {  
2   "from": "php:7.4-apache",  
3   "workdir": "/var/www/html",  
4   "copy": ". .",  
5   "run": "docker-php-ext-install mysqli pdo pdo_mysql",  
6   "expose": "80"  
7 }
```

4.2 compose-generator.php

Questo file riceve i dati in `$_POST` passati dal frontend con `submitHandler.js`, e li utilizza per generare il file Docker Compose corrispondente.

Inizialmente viene creato un file yml nella directory *Dockerfiles* presente nel backend. Successivamente viene aperto il file in modalità di scrittura, poi viene estratto il contenuto dell'array passato in `$_POST` contenente il testo che l'utente ha modificato sul modal per il Docker Compose. Questo contenuto viene poi assegnata alla stringa *txt*.

Alla fine dell'operazione, la stringa viene scritta sul file yml creato precedentemente. Dopo aver chiuso il file viene inviato il download tramite HTTP, utilizzando gli header *Content-type*, *Content-Disposition* e *Content-*

Length.

Dopo il download il file docker-compose.yml viene cancellato per conservare spazio di storage.

```
1  <?php
2  // Creazione file
3  $compose = "Dockerfiles/docker-compose.yml";
4  $file = fopen($compose, "w");
5
6  if (isset($_POST['dockerComposeContent'])) {
7      // Prendi il contenuto in post
8      $composeContent = $_POST['dockerComposeContent'];
9  }
10
11 // Aggiungi i contenuti alla stringa
12 $txt = "#Generato con Autodocker 2025!\n\n";
13 $txt .= $composeContent;
14
15 // Scrivi il contenuto sul file
16 fwrite($file, $txt);
17 fclose($file);
18
19 // Inizia il download
20 header('Content-Type: text/plain');
21 header('Content-Disposition: attachment; filename="docker-compose.yml"');
22 header('Content-Length: ' . filesize("$compose"));
23
24 // Invia il contenuto del file al browser
25 readfile("$compose");
26
27 // Cancella il file dopo aver mandato il download
28 unlink($compose);
29 ?>
```

4.3 dockerfile-generator.php

Questo file riceve i dati in \$_POST passati dal frontend con *submitHandler.js*, e li utilizza per generare il file Dockerfile corrispondente.

Inizialmente viene creato un file Dockerfile nella directory *Dockerfiles* presente nel backend. Successivamente, il codice itera sull'array di dati passato in \$_POST contenente l'input dell'utente sul form.

L'array passato è un array associativo con *chiave* → *valore*. La chiave è l'istruzione Dockerfile, mentre l'array contiene l'input dell'utente sul form.

Il codice itera sull'array in \$_POST e utilizza la funzione *addCommand(\$key, \$commands)* per aggiungere ogni comando alla stringa *txt*.

Dopo aver aggiunto tutti i comandi viene aperto il file creato precedentemente e viene scritto il contenuto di *txt*.

In seguito viene avviato il download tramite HTTP, utilizzando gli header *Content-type*, *Content-Disposition* e *Content-Length*.

Dopo il download il file Dockerfile viene cancellato per conservare spazio di storage.

```

1  <?php
2  // Scrive i comandi con piu' istanze sul file
3  function addCommand($commandType, $commands)
4  {
5      $commandsTxt = '';
6      foreach ($commands as $command) {
7          $commandsTxt .= strtoupper($commandType) . " " . $command . "\n";
8      }
9      $commandsTxt .= "\n";
10     return $commandsTxt;
11 }
12
13 // Creazione file
14 $dockerfile = "Dockerfiles/Dockerfile";
15
16 $txt = "#Generato con Autodocker 2025!\n\n";
17
18 // Aggiungi i comandi
19 foreach ($_POST as $key => $commands) {
20     $txt .= addCommand($key, $commands);
21 }
22
23 $file = fopen($dockerfile, "w");
24 fwrite($file, $txt);
25 fclose($file);
26
27 // Inizia il download
28 header('Content-Type: text/plain');
29 header('Content-Disposition: attachment; filename="Dockerfile"');
30 header('Content-Length: ' . filesize("$dockerfile"));
31
32 // Invia il contenuto del file al browser
33 readfile("$dockerfile");
34
35 // Cancella il file dopo il download
36 unlink($dockerfile);
37 ?>

```

5. Frontend – Struttura Sito

Questa sezione tratterà una breve descrizione della struttura del sito, comprendendo i file html.

5.1 Schermata Home (*index.html*)

Questo codice html costituisce la pagina home del sito, al cui interno è possibile avere informazioni sul funzionamento del sito e informazioni riguardanti DockerFile e Docker Compose.

La struttura del file è la seguente:

- **Navbar (*navbar-view.html*):** file composto da logo nella parte sinistra, scritta “Home” e scritta “DockerFile” per portare l’utente all’altra pagina del sito. Sulla parte destra è presente un pulsante per il cambio del tema.
- **Titolo e sottotitolo del sito.**
- **Cards (*home-cards.html*):** file composto da tre carte aventi la seguente struttura:
 - Icona raffigurante il contenuto della carta;
 - Breve frase per /parola per riassumere la qualità del sito;
 - Descrizione per approfondire e offrire maggiori informazioni riguardo la qualità sottolineata.
- **Descrizioni DockerFile/Docker Compose (*home-descriptions.html*):** file costituito da due blocchi con l’obiettivo di fornire documentazione sul significato e sull’utilizzo di DockerFile (primo blocco) e Docker Compose (secondo blocco).
Sono presenti nella parte sinistra di ciascun blocco domande e relative risposte riguardanti le due tipologie di file, mentre nella parte destra è presente un’immagine raffigurante un esempio di codice.
- **Footer (*footer-view.html*).**

5.2 Schermata Generazione File (*dockerfile-gen.html*)

Questo codice html costituisce la pagina per la generazione effettiva del DockerFile e del Docker Compose. Nella parte sinistra della pagina è presente un form, nel quale l'utente potrà inserire i comandi da scrivere all'interno del proprio DockerFile. Alla fine del form è presente una checkbox da spuntare per la generazione parallela del Docker Compose (che verrà generato a partire dai comandi inseriti dall'utente per il DockerFile) e il pulsante di conferma.

Nella parte destra della pagina invece è presente la preview del DockerFile (e del Docker Compose se la checkbox è stata spuntata dall'utente) in continuo aggiornamento a seconda dei comandi inseriti dall'utente. La struttura del file è la seguente:

- **Navbar (*navbar-view.html*):** file composto da logo nella parte sinistra, scritta "Home" per portare l'utente all'altra pagina del sito e scritta "DockerFile". Sulla parte destra è presente un pulsante per il cambio del tema.
- **Titolo della pagina.**
- **Form (*dockerfile-form-view.html*):** container contenente il form per la generazione del file avente la seguente struttura:
 - Pulsante per la selezione di un eventuale preset (*Nodejs*, *Flask* e *Apache*) per la creazione veloce del documento.
 - Campo del form per il comando FROM (obbligatorio) e relativo tooltip per una breve descrizione del comando.
 - Campi opzionali con relativo tooltip e pulsante per aggiungere lo spazio di Input.
 - Checkbox per generare anche il Docker Compose insieme al DockerFile.
 - Pulsante per la generazione del file (in caso di checkbox spuntato, questo pulsante porterà l'utente ad una schermata per eventuali modifiche al Docker Compose).
- **Preview File (*dockerfile-preview-view.html*):** file contenente la preview del DockerFile che riflette i cambiamenti che avvengono all'interno del Form. Se l'utente decide di generare anche il Docker Compose, compare anche la preview del Docker Compose con le stesse caratteristiche della precedente.
- **Footer (*footer-view.html*).**

6. Frontend – JavaScript

Questa sezione tratterà i file presenti in *frontend/script*.

6.1 index-presenter.js

Questo script utilizza *AJAX* per recuperare le html presenti in *frontend/assets/html* per poterli caricare dinamicamente sulla pagina *index*.

La funzione *appendContent* prende come input *tag* e *filename*, i quali sono rispettivamente l'id del container a cui aggiungere il contenuto e il path dell'html da appendere.

appendContent viene chiamata solo dopo che la pagina di *index* sia stata caricata completamente. Questo viene controllato con la funzione *\$(document).ready(function() { ... })*.

Viene prima caricata la navbar, poi le home-cards, le descrizioni ed infine il footer.

```

1 // Aggiungi il file indicato
2 function appendContent(tag, filename) {
3     $.ajax({
4         url: filename,
5         success: function (data) {
6             $(tag).append(data);
7         },
8         error: function () {
9             console.log("Errore nel caricamento del file: " + filename);
10        }
11    });
12 }
13
14 $(document).ready(function () {
15     appendContent("#navbar-container", "../assets/html/navbar-view.html"); //
    Carica la navbar
16     appendContent("#cards-container", "../assets/html/home-cards.html"); //
    Carica le tre cards
17     appendContent("#descriptions-container", "../assets/html/home-descriptions
    .html"); // Carica le due descrizioni
18     appendContent("#footer-container", "../assets/html/footer-view.html"); //
    Carica il footer
19 });

```

6.2 dockerfile-gen-presenter.js

Questo script si comporta come quello spiegato precedentemente; utilizza AJAX per recuperare le html presenti in *frontend/assets/html* per poterli caricare dinamicamente sulla pagina *index*.

È presente la stessa funzione *appendContent* che in questo caso carica la navbar, il form, la preview, il footer ed infine il modal per modificare il file compose.

Oltre ad aggiungere il contenuto alla pagina, nella chiamata *AJAX* vengono inizializzati i tooltip di bootstrap, utilizzati per mostrare descrizioni generali sui comandi del form.

```

1 // Aggiungi il file indicato
2 function appendContent(tag, filename) {
3     $.ajax({
4         url: filename,
5         success: function (data) {
6             $(tag).append(data);
7
8             // Inizializza tooltip
9             const tooltipTriggerList = document.querySelectorAll('[data-bs-
    toggle="tooltip"]');
10            const tooltipList = [...tooltipTriggerList].map(tooltipTriggerEl
    => new bootstrap.Tooltip(tooltipTriggerEl));
11        },
12        error: function () {
13            console.log("Errore nel caricamento del file: " + filename);
14        }
15    });
16 }
17
18 $(document).ready(function () {
19     appendContent("#navbar-container", "../assets/html/navbar-view.html"); //
    Carica la navbar
20     appendContent("#form-container", "../assets/html/dockerfile-form-view.html
    "); // Carica il form
21     appendContent("#preview-container", "../assets/html/dockerfile-preview-
    view.html"); // Carica la preview
22     appendContent("#footer-container", "../assets/html/footer-view.html"); //
    Carica il footer
23     appendContent("#dockercompose-edit", "../assets/html/dockercompose-edit.
    html"); // Carica il modal
24 });

```

6.3 sortable-container-init.js

Questo script ha il compito di inizializzare il container *sortable*, il quale permette all'utente la modifica dell'ordine dei comandi del form in modo dinamico.

Per fare utilizzo di questo tipo di container è stata importata la libreria [Sortable](#).

Per inizializzare il container, viene utilizzata `$(document).ready(function() { ... })` con `setTimeout(function () {}, 100)` per aspettare il caricamento completo della pagina insieme agli elementi aggiunti dinamicamente.

La funzione `setTimeout` cerca il container con la classe *sortable-container*. Quando viene trovato, la inizializza con le opzioni *animation* e *ghostClass* per gestire l'animazione del drag and drop.

Quando vengono trascinati gli elementi sul form, verrà stampato sul console il messaggio che indica che l'elemento è stato spostato sulla sua nuova posizione. Infine viene aggiornata la preview di Dockerfile per riflettere i cambiamenti avvenuti. Quest'ultima funzione viene importata dallo script *inputHandler.js*.

```
1 import { updateDockerfilePreview } from './inputHandler.js';
2
3 $(document).ready(function () {
4
5     // Aspetta caricamento DOM, poi inizializza Sortable
6     setTimeout(function () {
7         let formContainer = document.querySelector('.sortable-container');
8
9         if (formContainer) {
10             Sortable.create(formContainer, {
11                 animation: 150,
12                 ghostClass: 'sortable-ghost',
13                 onEnd: function (evt) {
14                     console.log("Spostato l'elemento", evt.oldIndex, "a", evt.
15 newIndex);
16                     updateDockerfilePreview();
17                 }
18             });
19         } else {
20             console.error("sortable-container non esiste!");
21         }
22     }, 100);
23 });
```

6.4 inputHandler.js

Questo script contiene le funzioni principali utilizzate per gestire gli input dell'utente.

- `addInputs(type, placeholder, defaultValue = '')`

Questa funzione ha il compito di inserire i campi di input per ogni comando che vuole aggiungere l'utente.

È una funzione con la keyword *export*, indicando che verrà utilizzata in altre script esterne.

Prende in input il tipo di comando che si vuole inserire, la scritta da inizializzare per il placeholder, e il valore dell'input. Quest'ultimo è sempre vuoto di default poiché viene utilizzato solo nel caso in cui l'utente voglia fare uso dei preset forniti.

La funzione prende come target i container con le id `#{type}Commands`. Successivamente viene effettuato il controllo per i comandi *CMD*, *ENTRYPOINT* e *STOPSIGNAL* dato che questi possono avere soltanto al massimo un campo di input. Per gli altri campi il default viene impostato a 5. Si raccomanda comunque all'utente di usare il minor numero di campi possibili per ridurre il numero di layer nell'immagine finale.

Dopo i controlli viene creata la variabile *newInput* la quale contiene la div per il campo di input e il tasto per rimuoverlo. Dopodiché si inserisce questa div al container preso come target in precedenza.

Infine vengono aggiornate le due preview all'input nel nuovo campo aggiunto.

Viene anche chiamata *updateComposePreview()* alla fine per aggiornare la preview del compose all'aggiunta di un nuovo campo di input.

```
1 // Aggiungi piu' istanze del comando
2 export function addInputs(type, placeholder, defaultValue = '') {
3   let containerId = `#${type}Commands`;
4   let name = type;
5
6   if (type === 'cmd' ||
7       type === 'entrypoint' ||
8       type === 'stopsignal') {
9     let currentInputs = $('#${containerId} .input-group').length;
10    if (currentInputs >= 1) {
11      alert('Si puo' avere solo un'istanza di "${type.toUpperCase()}"');
12    }
13    return;
14  } else {
15    let currentInputs = $('#${containerId} .input-group').length;
16    if (currentInputs >= 5) {
17      alert('Per ridurre il numero di layer si possono aggiungere solo $
18 {getMaxIterations()} istanze di "${type.toUpperCase()}"');
19      return;
20    }
21  }
22
23  let newInput = $('<div class="input-group mt-2">')
24    .append('<input type="text" class="form-control" name="${name}[]" "
25 placeholder="${placeholder}" value="${defaultValue}" required>')
26    .append('<button type="button" class="btn btn-danger remove-btn">
27 Rimuovi</button>');
28
29  $('#${containerId}').append(newInput);
30  $('input[name='${name}[]']').on("input", updateDockerfilePreview);
31  $('input[name='${name}[]']').on("input", updateComposePreview);
32
33  updateComposePreview();
34 }
```

- *updateDockerfilePreview()*

Questa funzione ha il compito di aggiornare la preview-box di Dockerfile per riflettere i cambiamenti effettuati dall'utente.

È una funzione con la keyword *export*, indicando che verrà utilizzata in altre script esterne.

Inizialmente viene creata una stringa *dockerfile* in cui viene scritta il comando FROM e il valore che l'utente ha inserito nel suo campo di input. Questo avviene sempre poiché ogni Dockerfile deve obbligatoriamente iniziare con il comando FROM.

Viene definita una funzione *appendCommands* la quale ha il compito di aggiungere i comandi sulla stringa *dockerfile* quando viene passato in input il nome del comando indicato.

Utilizzando il nome passato, trova il campo di input per quel comando e inserisce ogni input dentro la stringa. A questo punto la booleana *hasCommands* viene settata a true.

Alla fine della funzione viene effettuato il controllo su *hasCommands*. Questo è necessario per verificare che esistano input per quel tipo di comando passato in input. Se esiste, allora aggiunge una *newline* alla stringa. In questo modo non vengono aggiunte per i comandi senza input.

updateDockerfilePreview poi itera sui contenuti nel container sortable, e chiama *appendCommands* per aggiungere i comandi quando sono presenti input.

Infine viene scritto il testo di *dockerfile* sul container con id *dockerfilePreview*.

```
1 // Aggiorna la preview per Dockerfile
2 export function updateDockerfilePreview() {
3   //console.log($("#dockerfilePreview"));
4   let dockerfile = "FROM " + $("#from").val() + "\n\n";
5
6   // Funzione generale per aggiungere piu' istanze di comandi
7   function appendCommands(inputName) {
8     let command = inputName.toUpperCase();
9     let hasCommands = false; // Traccia se esistono comandi per questo
    tipo
10
11     $("input[name='" + inputName + "']").each(function () {
12       dockerfile += command + " "; // Aggiunge il comando
13       hasCommands = true; // Esistono comandi di questo tipo, allora
    setto a true
14       dockerfile += $(this).val() + "\n"; // Inserisce il valore
15     });
16
17     if (hasCommands) {
18       dockerfile += "\n"; // Vai a capo solo quando esistono comandi
19     }
20   }
21   // Itera sui contenuti in sortable-container e aggiunge i comandi
22   $('#sortable-container > div').each(function () {
23     let commandType = $(this).attr('id'); // Prendi il tipo di comando
    dall'id
24
25     // Aggiungi in base all'id
26     switch (commandType) {
27       case 'labelCommands':
28         appendCommands('label');
29         break;
30       case 'argCommands':
31         appendCommands('arg');
32         break;
33       case 'envCommands':
34         appendCommands('env');
35         break;
36       case 'healthcheckCommands':
37         appendCommands('healthcheck');
38         break;
39       ...
40     }
41   });
42
43   $("#dockerfilePreview").text(dockerfile);
44 }
```

- *updateComposePreview()*

Questa funzione ha il compito di aggiornare la preview-box di Docker Compose per riflettere i cambiamenti effettuati dall'utente.

È una funzione con la keyword *export*, indicando che verrà utilizzata in altre script esterne.

Inizialmente viene creata la stringa *composeFile* in cui vengono inserite le informazioni principali che di solito sono sempre uguali per i file Docker Compose.

Anche qui viene definita una funzione *appendCommands*, che però funziona in modo diverso da quella in *updateDockerfilePreview()*. In questo caso crea un array *items* per contenere il valore di input del comando passato. Per ogni campo di input, viene controllato che non sia vuoto prima di aggiungere il valore inserito nell'array *items*. Vengono rimossi i caratteri `[ ]` dal valore passato in input.

Infine viene restituito l'array *items*.

Viene definita un'altra funzione chiamata *appendSection* che ha il compito di aggiungere le diverse sezioni presenti nel Docker Compose come *volumes*, *ports*, *env* ed *expose*.

Prende in input il nome della sezione da aggiungere e l'array *items* per quel comando. Se l'array contiene qualcosa, viene aggiunta la nuova sezione, poi vengono scritti su *composeFile* i comandi trovati iterando sull'array.

updateComposePreview segue chiamando queste due funzioni per i comandi *VOLUME*, *COPY*, *ADD*, *ENV*, *EXPOSE*. È stato possibile aggiungere anche gli altri comandi del Dockerfile, ma è stato preferito aggiungere solo questi per renderlo più semplice, dato che Docker Compose sovrascrive i comandi presenti sul Dockerfile.

Gli input in *VOLUME*, *COPY*, *ADD* vengono formattati in un certo modo per Docker Compose. *ENV* ed *EXPOSE* vengono lasciati uguali, ma per *EXPOSE* viene aggiunta anche la sezione *ports* nel caso in cui l'utente abbia necessità di mappare le porte indicate.

Infine viene scritto il testo di *composeFile* sul container con id *dockercomposePreview*.

```
1 export function updateComposePreview() {
2   let composeFile = "version: '3.8'\n\nservices:\n  app:\n    build:\n      context: .\n      dockerfile: Dockerfile\n";
3
4   // Funzione generale per raccogliere piu' istanze di comandi
5   function appendCommands(inputName) {
6     let items = [];
7
8     $("input[name='" + inputName + "[]']").each(function () {
9       let value = $(this).val().trim();
10      if (value !== '') {
11        // Rimuovi eventuali caratteri [ ] e " dalle stringhe
12        value = value.replace(/[\[\]" "]/g, '');
13        items.push(value);
14      }
15    });
16
17    return items;
18  }
19
20  // Funzione per creare e aggiungere sezioni generiche
21  function appendSection(sectionName, items) {
22    if (items.length > 0) {
23      composeFile += "  " + sectionName + ":\n";
24      items.forEach(function (item) {
25        composeFile += "    " + item + "\n";
26      });
27    }
28  }
```

```

1 // Aggiungi Volumi
2 let volumes = appendCommands('volume');
3 volumes = volumes.map(volume => `.${volume}:${volume}`);
4
5 // Aggiungi COPY come volumi
6 let copies = appendCommands('copy');
7 copies.forEach(copy => {
8   let [source, destination] = copy.split(' ', 2); // Separa la sorgente
   dalla destinazione
9   volumes.push(`${source}:${destination}`);
10 });
11
12 // Aggiungi ADD come volumi
13 let adds = appendCommands('add');
14 adds.forEach(add => {
15   let [source, destination] = add.split(' ', 2); // Separa la sorgente
   dalla destinazione
16   volumes.push(`${source}:${destination}`);
17 });
18
19 appendSection('volumes', volumes);
20
21 // Aggiungi env e expose come prima
22 let envVars = appendCommands('env');
23 appendSection('environment', envVars);
24
25 let exposedPorts = appendCommands('expose');
26 appendSection('expose', exposedPorts);
27
28 // Se EXPOSE stato aggiunto, aggiungi anche la sezione PORTS
29 if (exposedPorts.length > 0) {
30   composeFile += "    ports:\n";
31   exposedPorts.forEach(function (port) {
32     composeFile += "        - \"" + port + ":" + port + "\"\n";
33   });
34 }
35
36 // Visualizza l'anteprima del file docker-compose.yml
37 $("#dockercomposePreview").text(composeFile);
38 }

```

- `resetForm()`

Questa funzione ha il compito di resettare il form, cancellando tutti gli input e i campi di input inseriti. Utilizza come target gli elementi che contengono `input[type="text"]` e imposta il loro valore ad una stringa vuota. Questo è utile in particolare per il campo FROM che ha sempre un campo di input aperto.

Successivamente viene utilizzato come target gli elementi con classe `input-group`, i quali sono soltanto i campi di input aggiunti dall'utente. Questi vengono rimossi con il `.remove()`.

Infine vengono aggiornate le due preview.

```

1 // Resetta il form cancellando tutti gli input e i campi di input
2 export function resetForm() {
3   // Cancella tutti gli input
4   $('input[type="text"]').val('');
5
6   // Togli tutte le div per input
7   $('.input-group').remove();
8
9   // Aggiorna le preview
10  updateDockerfilePreview();
11  updateComposePreview();
12 }

```

6.5 submitHandler.js

Questo script ha il compito di gestire l'invio dei dati del form in backend utili per generare Dockerfile e docker-compose.yml.

Vengono impostati come valori globali *my_server* e i diversi url necessari per rimandare *AJAX* ai file php nel backend. Quando il nome del server viene cambiato da *localhost* a qualcos'altro, è necessario cambiare il valore di *my_server*.

Viene definita la funzione *sendToBackend* il quale gestisce l'invio dei dati al backend. Prende in input l'url a cui inviare i dati, i dati da inviare e il nome del file che voglio avere per il download.

Utilizza *AJAX* per inviare i dati in *POST*. Viene impostato il tipo di risposta del xhr a *blob* per indicare che ci sarà un file per download.

Quando la chiamata *AJAX* ha successo viene creato un blob partendo dai dati del form. Successivamente viene generato l'url che riporta al blob che verrà cliccato automaticamente per far partire il download. Infine viene tolto il link.

In caso di errore vengono stampati sulla console i dati necessari per rilevare l'errore, e viene attivata un>alert per l'utente.

submitHandler controlla se la pagina è stata caricata completamente con i rispettivi elementi utilizzando le funzioni *\$(document).ready(function () {...})* e *setInterval(function () {...}, 100)*.

Viene inizializzato l'event handler per il tasto *submit* del form, il quale serializza i dati del form per poterli inviare con *AJAX*. Successivamente si effettua il controllo per vedere se il checkbox per Docker Compose sia stato spuntato o meno. In caso positivo viene aperto il modal per effettuare le modifiche sul *Docker Compose*. Viene copiato il contenuto di *dockercomposePreview* sul testo del modal.

Infine viene inizializzato l'event handler per il tasto *generateDockerCompose* il quale prende il contenuto del modal ed effettua le chiamate a *sendToBackend* per inviare i dati ed avviare il download di *Dockerfile* e *Docker Compose*.

In caso negativo viene effettuato solo il download del *Dockerfile*.


```

1 // Cambiare quando non si e' piu' in localhost
2 const my_server = 'localhost';
3
4 const main_url = 'http://${my_server}:8080';
5 const dockerfile_url = 'http://${my_server}:8080/dockerfile-generator.php';
6 const compose_url = 'http://${my_server}:8080/compose-generator.php';
7
8 // Invia i dati del form al backend con ajax per ottenere file download
9 function sendToBackend(url, formData, name) {
10     $.ajax({
11         url: url, // url dove inviare la richiesta
12         type: 'POST', // Invio a backend
13         data: formData, // Input del form
14         xhrFields: {
15             responseType: 'blob' // La risposta sara' un blob per il download
16         },
17         success: function (data) {
18             //console.log("AJAX Success:", data);
19             // Crea un blob partendo dai dati del form
20             var blob = new Blob([data], { type: 'text/plain' });
21
22             // Crea un link nascosto per il download e attivalo
23             var downloadUrl = window.URL.createObjectURL(blob);
24             var a = document.createElement("a");
25
26             // Nome del file
27             var filename = name;
28
29             // Download del file
30             a.href = downloadUrl;
31             a.download = filename;
32             document.body.appendChild(a);
33             a.click();
34
35             // Togli il link
36             window.URL.revokeObjectURL(downloadUrl);
37             a.remove();
38         },
39         error: function (xhr, status, error) {
40             console.error("AJAX Error - Status:", status);
41             console.error("AJAX Error - Message:", error);
42             console.error("AJAX Error - Response:", xhr.responseText);
43             alert('Errore con la richiesta AJAX!');
44         }
45     });
46 }

```

```

1 $(document).ready(function () {
2     // Controlla se il form e' stato inserito nella pagina ogni 100ms finche'
   viene trovato
3     var formExists = setInterval(function () {
4         if ($('#dockerfileForm').length) {
5             //console.log("Form trovato!");
6
7             // Azione tasto submit del form
8             $('#dockerfileForm').submit(function (e) {
9                 e.preventDefault();
10
11                 // Ottieni i dati del form
12                 var formData = $(this).serialize();
13                 //console.log(formData);
14
15                 // DockerCompose insieme al DockerFile
16                 if ($('#composeCheckbox').is(':checked')) {
17                     $('#dockerComposeModal').modal('show');
18
19                     // Popola il contenuto del modal con il contenuto di #
   dockercomposePreview
20                     var dockerComposeContent = document.getElementById('
   dockercomposePreview').textContent;
21                     document.querySelector('.modal-p').textContent =
   dockerComposeContent;
22
23                     // Associa l'event handler al tasto generateDockerCompose
24                     $('#generateDockerCompose').off('click').on('click',
   function() {
25                         // Prendi il contenuto modificato
26                         var modifiedContent = document.querySelector('.modal-p
   ').textContent;
27                         //console.log(modifiedContent);
28                         // Avvia download Dockerfile
29                         sendToBackend(dockerfile_url, formData, "Dockerfile");
30                         // Avvia download docker-compose
31                         sendToBackend(compose_url, { dockerComposeContent:
   modifiedContent }, "docker-compose.yml");
32                     });
33                     } else {
34                         // Avvia download Dockerfile
35                         sendToBackend(dockerfile_url, formData, "Dockerfile");
36                     }
37                 });
38
39                 // Ho trovato il form e ho eseguito tutto, non controllare piu'
40                 clearInterval(formExists);
41             } else {
42                 console.log("Non ho trovato il form...");
43             }
44         }, 100);
45     });

```

6.6 buttonActions.js

Questo script inizializza tutti gli event handler per ogni tasto presente sul form. Importa tutte le funzioni necessarie da *inputHandler.js*.

Anche qui vengono definite le variabili *my_server* e *main_url* per trovare i percorsi necessari per i preset nel backend.

Tutti gli event handler vengono inizializzati soltanto dopo *\$(document).ready(function () {...})* per controllare che la pagina sia stata caricata completamente.

Per l'input su FROM vengono chiamate le funzioni *updateDockerfilePreview()* e *updateComposePreview()* per aggiornare le preview.

Per gli altri comandi viene chiamata soltanto *addInputs()* passando il tipo di comando e il placeholder per il campo d'input.

Viene gestito anche l'event handler per il tasto per rimuovere campi. In questo caso rimuove il div corrente in cui si trova, e successivamente aggiorna le due preview.

Per poter inserire il contenuto dei preset nei campi di form, vengono inizializzati gli event handler per i tasti di nodeJS, Flask ed Apache. Effettuano tutti una chiamata *AJAX* per ottenere i dati dei file JSON rispettivi nel backend. Prima di aggiornare i valori di ogni campo viene fatta una chiamata a *resetForm()* per cancellare tutti gli input precedenti. In questo modo si evitano di duplicare i comandi quando vengono premuti i tasti più di una volta.

```
1  import { updateDockerfilePreview, addInputs, updateComposePreview, resetForm }
   from './inputHandler.js';
2
3  // Cambiare quando non si e' piu' in localhost
4  const my_server = 'localhost';
5
6  const main_url = 'http://${my_server}:8080';
7
8  $(document).ready(function () {
9      // Aggiorna FROM
10     $(document).on("input", "#from", function () {
11         updateDockerfilePreview();
12         updateComposePreview();
13     });
14
15     // Aggiungi LABEL
16     $(document).on('click', '#addLabelbtn', function () {
17         addInputs("label", "es: maintainer=nome.cognome@email.com")
18     });
19
20     // Aggiungi ARG
21     $(document).on('click', '#addArgbtn', function () {
22         addInputs("arg", "es: VERSION=1.0")
23     });
24
25     // Aggiungi ENV
26     $(document).on('click', '#addEnvbtn', function () {
27         addInputs("env", "es: APP_ENV=production")
28     });
29
30     ... // Uguale per gli altri comandi
31
32     // Tasto per rimuovere input
33     $(document).on('click', '.remove-btn', function () {
34         $(this).parent().remove();
35         updateDockerfilePreview();
36         updateComposePreview();
37     });
```

```

1 // Preset NodeJS
2 $(document).on('click', '#nodejsBtn', function () {
3     // Prendi i dati del file
4     $.ajax({
5         url: '${main_url}/dockerfileTemplates/nodejs.json',
6         type: 'GET',
7         dataType: 'json',
8         success: function (data) {
9
10            // Resetta il form
11            resetForm();
12
13            // Inserisci i value
14            $('#from').val(data.from);
15            addInputs("workdir", "es: /dir", data.workdir);
16            addInputs("copy", "es: ./test.txt /dir/", data.copy);
17            addInputs("run", "es: apt-get update -y", data.run);
18            addInputs("expose", "es: 80", data.expose);
19            addInputs("cmd", '["CMD", "curl", "--fail", "http://mio-sito.com", "|", "exit", "1"]', data.cmd);
20
21            // Aggiorna preview
22            updateDockerfilePreview();
23            updateComposePreview();
24        },
25        error: function (error) {
26            console.error('Error fetching JSON:', error);
27        }
28    });
29 });

```

```

1 // Preset Apache
2 $(document).on('click', '#apacheBtn', function () {
3     // Prendi i dati del file
4     $.ajax({
5         url: '${main_url}/dockerfileTemplates/apache.json',
6         type: 'GET',
7         dataType: 'json',
8         success: function (data) {
9
10            // Resetta il form
11            resetForm();
12
13            // Inserisci i value
14            $('#from').val(data.from);
15            addInputs("workdir", "es: /dir", data.workdir);
16            addInputs("copy", "es: ./test.txt /dir/", data.copy);
17            addInputs("run", "es: apt-get update -y", data.run);
18            addInputs("expose", "es: 80", data.expose);
19
20            // Update previews
21            updateDockerfilePreview();
22            updateComposePreview();
23        },
24        error: function (error) {
25            console.error('Error fetching JSON:', error);
26        }
27    });
28 });
29 });

```

6.7 pageTheme.js

Questo script permette all'utente di cambiare lo stile della pagina, passando da un tema chiaro a un tema scuro, e viceversa.

Questo è possibile cliccando sull'icona presente nella parte a destra della navbar.

Per ogni componente grafico da modificare, sono presente nel file css degli attributi per il relativo tema. Quando il pulsante viene premuto, i componenti adotteranno gli attributi appropriati.

```
1  const button = document.getElementById("theme-toggle");
2  const body = document.body;
3
4  // Controlla se il tema salvato in localStorage è "dark" e lo applica
5  if (localStorage.getItem("theme") === "dark") {
6    body.classList.add("dark-mode"); // Aggiunge la classe per il tema scuro
7    button.textContent = "Chiaro";
8  }
9
10 // Aggiunge un event listener per il click sul pulsante
11 button.addEventListener("click", () => {
12   body.classList.toggle("dark-mode");
13
14   // Il tema scuro viene salvato nel local storage
15   if (body.classList.contains("dark-mode")) {
16     localStorage.setItem("theme", "dark");
17     button.textContent = "Chiaro";
18   } else { // Il tema chiaro viene salvato nel local storage
19     localStorage.setItem("theme", "light");
20     button.textContent = "Scuro";
21   }
22 });
```